

ELEC 278

Report on Data Structure Design Choices and Algorithm Complexities

Electrical and Computer Engineering
Faculty of Engineering and Applied Science
Queen's University

Prepared by:
[Daniil Nistribenko]

[3/12/2024]

Introduction

The report outlines the design choices made in implementing key functionalities of a social media platform, focusing on the data structures used to manage Users, Friends, Messages, and Posts. For each data structure, the report discusses the selection rationale, structural details, modifications made by the implemented functions, and the methods used for searching and reading data.

1. User Profile Management

1.1 Description of the core data structure for User Profile Management

A hash table manages user profiles, with each user stored using their name as the key. The User structure includes fields like user_id, name, email, and a linked list of friends. A hash function computes an index, and collisions are handled through separate chaining, where users hashing to the same index are linked in a list. This structure enables efficient insertion and retrieval of profiles by name.

1.2 Trade-offs and Complexity

A hash table was chosen for managing user profiles due to its average-case $O(1)$ time complexity for insertion and search, efficiently handling 10,000 users. Collisions, though rare, can cause worst-case $O(n)$ search times, where n is the number of users in a bucket. To minimize collisions, a custom hash function processes each character in a name, multiplying the hash value by 2 and adding the character's ASCII value, ensuring a uniform distribution of hash values. Table 1 compares this solution with other data structures. Table 1 below provides a comparison of the hash table solution with other data structures.

Table 1: Pros and Cons comparison for each Data Structure for User Profile Management

Data Structure	Pros	Cons
Linked list	Simple implementation, dynamic Size	$O(n)$ search time complexity
Hash Table	Average $O(1)$ search and insertion	Potential for collisions, extra memory for linked lists
Binary Search Tree	Search $O(\log(n))$ if balanced	Complex implementation, required balancing mechanisms
Queue	FIFO operation, easy to implement	Not appropriate for random access or efficient searches
Stack	LIFO operation, easy to implement	Not effective for random access or efficient searches

1.3 Description of the fields individual structure

The User structure represents individual profiles, including fields for unique identification and relationships. The int user_id is a unique integer assigned to each user. The char name[NAME_LENGTH] stores the user's name as a fixed-size character array, while the char email[EMAIL_DOMAIN_LENGTH] holds their email address similarly. The FriendNode* friends is a pointer to a linked list, representing friendships, where each node links to another user.

1.4 Data Structure Modifications by Functions

Several functions manage user profiles and friendships in the data structure. The create_user(const char* name, const char* email) function initializes a User with a unique user_id, inserts it into the hash table, and adds it to the linked list at the index determined by hashing the name. The change_user_name(User* user, char* new_name) function updates a user's name by removing them from the old hash index and reinserting them at the new index. The change_user_email(User* user,

char* new_email) function updates the email field without altering the user's position in the hash table. The delete_user(User* user) function removes the user from the hash table and linked list while ensuring all references to the deleted user are removed from friends' lists to prevent dangling pointers.

1.5 Data Structure Search and Read Operations

The data structure includes several functions for searching, reading, and managing user profiles. The search_user_by_name(const char* name) function locates a user by calculating the hash index of the name, traversing the linked list at that index, and returning the User pointer if a match is found, or NULL otherwise. The search_user_by_email(const char* email) function iterates through all buckets in the hash table, traversing their linked lists to find a user by email, returning the corresponding User or NULL if not found. The print_users() function collects all user names from the hash table, sorts them alphabetically, and prints the list. Similarly, the print_friends(User* user) function traverses a user's friends list, sorts their names alphabetically, and displays them. A helper function, is_name_or_email_taken(const char* name, const char* email), ensures unique names and emails by checking all buckets and linked lists, returning 1 if a match is found and 0 otherwise.

2. Friend Management

2.1 Description of the core data structure for Friend Management

For managing friendships between users, a singly linked list is used within each User structure. Each user has a friends field, which is a pointer to a linked list of FriendNode structures.

Each FriendNode represents a friendship with another user by containing a pointer to that user's User structure. Thus, the approach allows each user to maintain a dynamic list of friends, enabling efficient addition and removal of friends.

2.2 Trade-offs and Complexity

The linked list data structure was chosen for managing friends due to its dynamic nature and ease of insertion and deletion operations. Each user's friends list can grow or shrink as friendships are added or removed, without the need for resizing or reallocating memory.

The main trade-off of using a linked list is the $O(n)$ time complexity for search operations, where n is the number of friends a user has. This means that checking if a user is a friend involves traversing the list, which can be inefficient for users with a large number of friends. However, given that the average number of friends per user is expected to be moderate, the trade-off is acceptable.

Additionally, Table 2 below provides a comparison each data structure.

Table 2: Pros and Cons of Data Structure for Friend Management

Data Structure	Pros	Cons
Singly Linked list	Dynamic Size, easy insertion and deletion	$O(n)$ search time complexity
Doubly Linked list	Traversal in both direction, head to tail, tail to head	Extra memory overhead for additional pointers
Binary Search Tree	$O(\log(n))$ search if balanced	Complex implementation, requires balancing
Hash Set	Average $O(1)$ search, efficient lookup	Additional memory overhead, complexity

2.3 Description of the Fields of Individual Structure

The FriendNode structure represents a single friendship, linking users through its fields. The User* user is a pointer to the User structure of the friend, providing direct access to their details and enabling

operations like traversing the friends list or managing friendships. The `FriendNode*` `next` is a pointer to the next node in the linked list, connecting all friends of a user and allowing sequential traversal of the list.

2.4 Data Structure Modifications by Functions

The friends linked list in each `User` structure is managed through several functions. The `add_friend(User* user1, User* user2)` function establishes a bidirectional implementation by creating and adding `FriendNode` entries for each user in the other's friends list. The `delete_friend(User* user1, User* user2)` function removes the friendship by traversing and deleting the `FriendNode` entries linking the two users in their respective lists. When a user is removed, the `delete_user(User* user)` function ensures no dangling pointers remain by iterating through all friends lists and deleting any `FriendNode` referencing the removed user.

2.5 Data Structure Search and Read Operations

Several functions handle searching and reading the friends linked list to provide key functionalities. The `print_friends(User* user)` function traverses a user's friends list, collects names in an array, sorts them alphabetically using bubble sort, and prints the sorted list. The `mutual_friends(User* user1, User* user2)` function identifies mutual friends by comparing user1's friends list against user2's, collecting matches into an array or hash set. The `print_mutual_friends(User** friends)` function takes the array of mutual friends, sorts it, and prints their names. Additionally, the `are_friends(User* user1, User* user2)` function checks if user2 is in user1's friends list by traversing the list and returning true if a match is found, supporting features like a messaging system.

3. Messages

3.1 Description of the core data structure for Message Management

Messages between users are managed using a combination of a doubly linked list for storing messages within individual chats and a singly linked list for organizing multiple chats. Each conversation between two users is represented by a `Chatstructure`, which maintains a doubly linked list of `Message` structures. This setup ensures that messages within a chat are stored in sequential order, allowing traversal from the oldest to the newest or vice versa. To manage multiple conversations, chats are stored in a singly linked list of `ChatNode` structures. Each `ChatNode` contains a `Chat` structure and a pointer to the next `ChatNode`, enabling the system to dynamically handle an arbitrary number of chats between different user pairs.

3.2 Trade-offs and Complexity

The choice of a doubly linked list for storing messages within a chat provides efficient insertion and deletion operations, particularly since messages are added to the end of the chat. Thus, make the structure beneficial for First-In-First-Out (FIFO) order.

Additionally, **Error! Reference source not found.** below provides a comparison each data structure.

Table 3: Pros and Cons for Data Structure for Messages

Data Structure	Pros	Cons
Singly Linked list	Dynamic size, easy insertion at the end	No backward traversal; less efficient deletion
Doubly Linked list	Dynamic size, bidirectional traversal	Extra memory overhead for additional pointers

Hash Table	Average $O(1)$ search, efficient lookup	Not necessary for sequential message storage
Queue	FIFO operation; simple implementation	Limited flexibility; no backward traversal

3.3 Description of the Fields of Individual Structure

The Message structure represents individual messages exchanged between users, designed to support First-In-First-Out (FIFO) order. Each message has a unique `int message_id` for identification. The `User*` sender and `User*` receiver pointers link the message to its sender and recipient, while the `char content[MAX_MESSAGE_LENGTH]` stores the message's text, ensuring it doesn't exceed the maximum length. The `Message*` prev and `Message*` next pointers form a doubly linked list, enabling backward and forward traversal within a chat. The Chat structure manages conversations between two users, with `Message*` head and `Message*` tail pointing to the first and last messages, respectively, and an `int size` tracking the number of messages, ensuring it doesn't exceed `MAX_MESSAGES`. The `ChatNode` structure links multiple chats in a singly linked list, with `Chat*` chat representing a conversation and `ChatNode*` next connecting to the next chat in the list.

3.4 Data Structure Modifications by Functions

The message management system dynamically handles messaging between users using a doubly linked list for storing messages within a chat and a singly linked list for managing multiple chats. Messaging starts with the helper function `get_chat(sender, receiver)`, which finds or creates a chat. To send messages, `are_friends(user1, user2)` ensures both users are friends (described above). Messages are then created and displayed in the chat. The `create_message(User* sender, User* receiver, const char* content)` function retrieves or creates a chat, initializes a new Message with unique `message_id`, sender and receiver pointers, and content, then appends it to the chat's messages list. If the chat exceeds `MAX_MESSAGES`, the oldest message is removed by updating the head pointer and freeing memory. The `display_chat(User* user1, User* user2)` function retrieves the chat and traverses its messages in FIFO order for display. When a user is deleted using `delete_user(User* user)`, chats involving the user become inaccessible, though messages remain in memory.

3.5 Data Structure Search and Read Operations

The message management data structures are searched and read through several key functions. The `get_chat(User* user1, User* user2)` helper function searches the `ChatNode` linked list to locate the chat between the two users, comparing participants by examining message senders and receivers. If no chat exists, it creates a new one, adds it to the list, and returns a pointer to it. The `display_chat(User* user1, User* user2)` function retrieves the chat, traverses its doubly linked list from head to tail, and sequentially prints each message for an organized view of the conversation. The `create_message(User* sender, User* receiver, const char* content)` function ensures that the sender and receiver are friends by invoking `are_friends(sender, receiver)`, validates that the message content does not exceed `MAX_MESSAGE_LENGTH`, and uses `get_chat` to find or create the appropriate chat. Data integrity is maintained through checks for friendship verification, content length validation, and assigning unique `message_ids` using a counter.

4. Posts

4.1 Description of the core data structure for Posts

The system uses a singly linked list to manage posts, represented by the Post structure. New posts are dynamically added to the front of the list. Each post maintains its own linked list of LikeNode structures to track users who have liked the post.

4.2 Trade-offs and Complexity

Post operations include `new_post` with a time complexity of $O(1)$ as posts are added to the front of the linked list, `add_like` at $O(n)$ due to checking existing likes among the users who liked the post, and `display_feed` at $O(n)$, driven by the sorting algorithm used to organize and display the collected posts. Below is Table 4, with analysis of the advantages and disadvantages, as well as the complexity of key operations.

Table 4: Data Structure Pros and Cons Comparisson for Post

Data Structure	Pros	Cons
Singly Linked List	Dynamic size for memory usage, $O(1)$ time complexity for insertion	Linear complexity for searching $O(n)$ or accessing specific post
Hash Table	$O(1)$ look up for likes	Higher memory usage and complex initialization
Doubly Linked List	Traversal in both directions	Memory overhead for extra pointers

4.3 Description of the Fields of Individual Structure

The Post structure represents individual posts. The `int post_id` field provides a unique identifier automatically generated for each post. The `User* user` pointer links the post to its creator, granting access to the user's details. The `char* content` stores the post's content as a dynamically allocated string, accommodating variable lengths within a specified maximum. The `int likes` tracks the total likes received, updating as new likes are added, while the `LikeNode* liked_by` points to the head of a singly linked list that records users who liked the post. Finally, the `Post* next` pointer connects posts in a singly linked list, supporting dynamic insertion and traversal.

4.4 Data Structure Modifications by Functions

Post-related operations modify the data structure as follows. The `new_post(User* user, const char* content)` function creates a new post and inserts it at the front of the singly linked list by updating the new post's next pointer to the current head and setting the head pointer to the new post, with a time complexity of $O(1)$. The `add_like(Post* post, User* user)` function adds a like to a post by traversing the `liked_by` list to ensure the user hasn't already liked it, then inserting a new `LikeNode` at the front if not found, and incrementing the likes count, with a time complexity of $O(n)$, where n is the number of likes. The `display_feed(User* user)` function collects posts from the user and their friends, sorts them by likes and recency, and displays them, with a time complexity of $O(n)$, where n is the total number of posts sorted.

4.5 Data Structure Search and Read Operations

The `search_post_by_content(const char* content)` function iterates through the posts linked list to find a post with matching content, with a time complexity of $O(n)$ due to worst-case traversal. The `display_feed(User* user)` function creates a feed by collecting posts from the user and their friends,

filtering and sorting them before display, with a time complexity of $O(n)$ when sorting. The `add_like(Post* post, User* user)` function checks the `liked_by` linked list to ensure a user hasn't already liked the post, adding a new like if not found, with a time complexity of $O(n)$ based on the number of likes.